Game Theory and the Dead Internet Theory:
Interaction between Bots and Humans on the Web

Controversies in Game Theory FS25

June 2025

1 Introduction

In recent years, an increasingly discussed idea has gained attention online — the Dead Internet Theory. The Dead Internet theory is a conspiracy theory which states that the Internet now consists mainly of bot activity and automatically generated content. It has two components: that human activity on the web has been replaced by algorithmically selected search results supervised by AI, and that the government is doing this to manipulate society. In this paper, we will focus more on the first part. This theory has gained more and more believers throughout the world in the last few years, as social media platforms have become the most popular form of news and information. Similar theories such as the Illuminati theory are also widely known. [1] We believe that if the Dead Internet Theory were to take place, trust in online interaction would worsen, as users would become more uncertain whether they are engaging with a person, a bot, or a cleverly trained AI agent.

Authenticity plays a key role in this scenario. Humans must signal their identity in ways that are credible, while bots try to mimic these signals to pass as genuine, with the objective of retaining the human's attention. But we people don't want to interact with fake news and posts, but preferably seek authentic and entertaining content that satisfies our needs. This interaction between humans and bots is what we call the Authenticity Signal Game, which we will look at in more detail throughout this report. Further, three domains of game theory (classical, evolutional, and behavioural) will be inspected and discussed in context with the Dead Internet Theory.
This topic is especially important for the future of social networks. One can visualize possible outcomes and different futures (dystopian or utopian) by simulating a signalling game and letting it run for many cycles.
The goal of this project is to find a good equilibrium within this game, where authentic content thrives and to look at what needs to be done for that to happen. Possibly to achieve this equilibrium, it may require external influences such as technology or policy. But we will mostly look at the analytical results of our simulations and some theoretical ideas within game theory.

2 Game Theory and the Dead Internet Theory

2.1 Classical Game Theory

In Game Theory, the dynamics and implications of the Dead Internet Theory have multiple intersections with it's domain. In classical game theory, the foundational domain, the focus is placed on analyzing rational decision-makers and their strategies usually leading to a conflict of interest [2]. Rationality in the context of game theory in general is a highly debated, almost enigmatic concept [3]; however, for this report we will simplify it to the following: The act of a person or a party of choosing a strategy that maximizes their own payoff given the other people's or parties' strategy/strategies.

There are 2 main actors in the case of the Dead Internet Theory: 1. Human users and 2. AI/Bot operators. While both are assumed to be rational, they have conflicting interests. Humans utility include authentic information, meaningful social connection, entertainment, etc., whereas bots objectives may range from maximizing engagement metrics for revenue, misinformation or controlling/spreading narratives (as per the Dead Internet Theory).
A Nash Equilibrium is a state where no player can improve their payoff by changing their own strategy, whilst all other players maintain their strategy. It is a stable point where everyone is doing their best response [4]. This concept will be elaborated further.

Although classical game theory provides a useful "framework" for analyzing situations, the complexity of interactions & dynamics between both actors in the Dead Internet Theory is heavily abstracted, which is why we consider the problem in other domains as well.

2.2 Evolutional Game Theory

In evolutionary game theory [5], strategies are not consciously chosen by individuals but rather traits of behaviours that are "inherited" either genetically or culturally, and whose prevalence changes based on their success. As such, strategies of both players align with their objectives. While from an evolutionary standpoint these may not have been relevant a few decades back, humans in the digital age may pursue "seeking authenticity", "engaging with superficial content" (for example through doom-scrolling), or even simply "disengaging from online platforms" entirely. While a bots strategy might not be entirely evident, we consider the strategies "mimicking human behaviour" and "mass-producing generic content" as this aligns with the goal mentioned above.
An Evolutionarily Stable Strategy (ESS) is a strategy, that once common in a population, cannot be successfully invaded by a new mutant strategy. This is due to a natural selection-dynamic as such alternative strategies either perform worse or fail to gain an advantage. In the context of the Internet and the Dead Internet Theory, this however begs the question which ESS's exist and hence also what are longterm consequences thereof?
If humans become complacent, i.e. "engage with superficial content" and don't effectively filter out poor quality, mass-produced AI content, then "mass-producing generic content" could become a stable strategy, as other strategies would become redundant due to its cost and effectiveness. Consequently, the Dead Internet scenario will quickly be established. If human users consistently prioritize authenticity and actively disengage with AI however, then this might force bots to pursue the "mimicking human behaviour" strategy. The coevolution could lead to a scenario where both sides constantly adapt, or to an ESS where mostly authentically-perceived content survives, be it from humans or from bots, making the internet at least feel more human, even if this may not be the case.

2.3 Behavioural Game Theory

When viewing the Dead Internet Theory through the lens of behavioural game theory, we attempt to consider the problems of abstractions of other game theoretical models which come with human behaviour, and the AI/bots adapting thereupon. In doing so, we attempt to bridge the gap between theory and reality by accounting for the psychological nuances of human decision-making.

We begin this section by questioning the human perception of authenticity, i.e. how do humans perceive what is authentic, what influence does this have over the Internet? What does authenticity even mean?

In psychology, authenticity is acting according to one's true self and behaving congruently with values and personality, as opposed to conforming to external, social pressures. [6] However this is to no one's surprise extremely subjective, as perceived authenticity can be heavily influenced e.g. through self-ratings & one's own authenticity. A paper [7] even states that no evidence was found which shows that people can accurately identify who is authentic, which can be advantageous, however also problematic in a digital environment. Human cognitive biases can be exploited, as humans tend to favor information confirming their existing beliefs. The bandwagon effect is a phenomenon which describes that humans are influenced by others and tend to simply follow what most people do. Furthermore, humans have preferences for fairness, reciprocity and genuine social connections, which heavily contribute to trust. All of these factors combined, with the prevalence of generative AI content and bot interactions can erode perceived authenticity and trust in online interactions.

If the Dead Internet Theory were to slowly take place, perceived authenticity could drop significantly, and consequently lead to an adaptation of human behaviour. Examples of such could be disengaging from social media, seeking out niche human-only communities, or generally just becoming extremely sceptical of all online content. This disengagement however, while protective for the individual, could lead to a negative cycle of a continuously increasing "deadness" of the internet, further supporting the Dead Internet Theory. The human perception of generative AI content on the other hand is for the most part pretty positive, as suggested by this study [8] : "The content generated by generative AI and augmented AI is perceived as of higher quality than that produced by human experts and augmented human experts. Second, revealing the source of content production (either human or AI) reduces – but does not reverse – the perceived quality gap between human- and AI-generated content. This bias in evaluation is predominantly driven by human favoritism rather than AI aversion: knowing the same content is created by a human expert increases its (reported) perceived quality, but knowing that AI is involved in the creation process does not affect its perceived quality."

Another key aspect of online interaction is imperfect information. Humans often don't know if content is human or AI-generated, thus resulting to incomplete or asymmetric information. A bot in this case has an information advantage, as it mostly knows the true origin of the content, therefore further allowing it to exploit human tendencies. This begs the question whether humans can cooperate with each other and resultingly collectively have a better payoff than acting individually.
In the following, we define zero-sum & non-zero-sum (general-sum) games and link it to the Dead Internet Theory. A zero-sum game is a game where the total winnings are equal to the total losses [9]. In the case of a two-player game, whatever one player wins, the other one loses. In the case of multiple players, the "payoff" for winners is equal to the combined loss of the losers. This is opposed to a non-zero-sum game, where a gain on a players end doesn't mean a loss on the other's end, meaning both players can technically "win". In the context of the Dead Internet Theory, a win simply means the successful pursuit of a strategy, which will be described in a payoff matrix later. However even the unsuccessful pursuit mustn't result in a general loss, which is why a non-zero-sum game is also of importance.

To end this subchapter we look at a publication exploring the constraints on artificial general intelligence [10]. Although its inquiries are presented in a highly theoretical manner, we still believe it shares useful information worth mentioning. The core argument is that for an AI to be considered "general", it needs to achieve superhuman performance not just in zero-sum games (where winning and losing are clearly defined), but also in general-sum games (non-zero-sum games), where such definitions are not always clear. The framework is built upon four key assumptions:

- Superhuman Machine (SHM): The machine (M) can outperform a human (H) in every general-sum game played between them.
- Machine Strategy (MS): The human (H) assumes the machine's (M) strategy is given.
- Rationality (R): The human (H) chooses a strategy that maximizes their own payoff given the machine's (M) strategy.
- Strategic Unpredictability (SU): The human (H) cannot be forced to follow any specific course of action predetermined by the machine (M).

The paper concludes an impossibility theorem, stating that under these four assumptions, when taken together, are inconsistent, i.e. it becomes impossible for the machine to consistently surpass the human in general-sum games.

3 Game Setup, Model & Python Simulation

Nowadays, nearly everyone engages in some sort of digital media. Be it YouTube, Instagram, or WhatsApp, most people are regularly exposed to digital platforms and other users. And between all human users, more and more bots come up. Some are designed to provide 24/7 support, answering questions even at 2 a.m. Other bots are not designed to help; instead, they try to lure you to fraudulent websites to steal your credit card information or identity. Some are built to engage with the common users on (e.g. dating) platforms to get money from you.
It is only understandable that in the depths of the web, some humans seek authenticity. Chatting with a bot can be nice and helpful, but sometimes you want to get to know someone new and find genuine love, instead of getting lovebombed by a bot.
The Authenticity Game is designed to model this socio-technical environment as a game-theoretic framework. It models the interactions between humans and bots with different scenarios and strategies. The game aims to explore possible strategies for reclaiming authenticity in a digital world dominated by noise, manipulation, and endless scrolling.

Each game is played by four players, two of type `Human` and two of type `Bot`. We define them as follows:
`Humans:`

H1 Humans that seek authenticity, that do not want to interact with bots

H2 Humans that engage with the flow, not caring if they interact with a human or a bot

`Bots:`

B1 Bots that try to mimic humanity and are therefore hard to detect by humans

B2 Bots that just mass produce, generating high-volume, low-quality content

The payoff matrix describes the outcome of each interaction between the parties and their respective strategies in the sense of a numerical reward. From some simple thought processes, we have created our own version of a payoff matrix for our exact scenario. This matrix remains constant for all regarded simulations in our game and is shown in Table 1:

	B1	B2
H1	(20, 10)	(30, 0)
H2	(0, 20)	(-20, 30)

Table 1: Payoff matrix: The first value in each cell corresponds to the payoff of the human player (H1 or H2), and the second to the bot player (B1 or B2).

Each game is played over multiple rounds. In every round, we choose at random one human and one bot according to their respective distributions. So for the humans it is either H1 or H2 and analogously B1 or B2 for the bots. We then take the corresponding payoffs from the payoff matrix and add these to the players' respective scores. The final cumulative score of each player decides who wins the game.
After all rounds are played, we create a plot to visually follow the outcomes of the game and each players' progression.

3.1 Simulation 1 - Base Game

This simulation presents the simplest version of the game of the interaction of humans and bots on the Internet, as explained in the section above. Here is a short summary of what the code exactly does:

The first 25 lines contain the imports used, numpy for the random picking (explained later) and the cumulative summing, used in Lines 58 and 59, and matplotlib.pyplot for the plotting of the games. In this section, we define the payoff matrix, the number of rounds which we can adapt, and the distributions of each type. In Line 27 we define our function `simulate_game` which takes as input an integer that represents the number of the game rounds (defined in section Game Analysis 3.4). Lines 30 and 31 initialize our scoring tracker that keeps track of the payoffs of each player. Lines 34 to 44 describe the actual game. In every round, we pick H1 or H2 and B1 or B2 at random [Line 36/37]. Then, in Line 40, we get the payoff based on the payoff matrix and update our scores/append the scores to our trackers. Lines 44 to 71 plot these values into a graph.

```python
import numpy as np
import matplotlib.pyplot as plt

#Human Users (H):
#   H1: Seek Authenticity
#   H2: Engage with Flow

#Bot Operators (B):
#   B1: Mimic Humanity
#   B2: Mass Produce

# Payoff-Matrix: [H1 vs B1, H1 vs B2, H2 vs B1, H2 vs B2]
payoffs = {
    ('H1', 'B1'): (20, 10),
    ('H1', 'B2'): (30, 0),
    ('H2', 'B1'): (0, 20),
    ('H2', 'B2'): (-20, 30)
}

# Number of rounds to simulate
rounds = 100

# Initial distribution (can be adjusted)
human_distribution = {'H1': 0.5, 'H2': 0.5}
bot_distribution = {'B1': 0.5, 'B2': 0.5}

def simulate_game(games):

    #Tracking scores
    human_scores = []
    bot_scores = []


    for _ in range(rounds):
        # Randomly pick strategies
        human = np.random.choice(['H1', 'H2'], p=[human_distribution['H1'], human_distribution['H2']])
        bot = np.random.choice(['B1', 'B2'], p=[bot_distribution['B1'], bot_distribution['B2']])

        # Get payoffs based on the chosen strategies
        h_payoff, b_payoff = payoffs[(human, bot)]

        # Update scores
        human_scores.append(h_payoff)
        bot_scores.append(b_payoff)

    # Plot results
    # Display the settings used in the simulation
    settings_text = (
        f"H1/H2: {human_distribution['H1']*100:.0f}% / {human_distribution['H2']*100:.0f}%\n"
        f"B1/B2: {bot_distribution['B1']*100:.0f}% / {bot_distribution['B2']*100:.0f}%\n"
    )

    # Adjusting the plot layout to include settings text
    plt.subplots_adjust(bottom=0.2)
    plt.gcf().text(0.01, 0.01, settings_text, fontsize=9, verticalalignment='bottom')

    # Plotting cumulative scores
    plt.plot(np.cumsum(human_scores), label='Human Payoff', color='red')
    plt.plot(np.cumsum(bot_scores), label='Bot Payoff', color='blue')

    # Adding labels and title to the plot
    plt.xlabel("Round")
    plt.ylabel("Cumulative Payoff")
    plt.title("Authenticity Signal Game     Simulation 1")          #Title displayed on the plot
    plt.suptitle(f"Number of Game: {games}" , fontsize=10)        #Subtitle displayed on the plot
    plt.legend()                                                  #adds legend to the plot
    plt.grid(True)                                                #adds grid to the plot
```

```


    plt.savefig(f"Results_Sim1/sim1_{games}.png")
    plt.clf()                                                   #deletes plot for next simulation

    # plt.show()                                                #shows the picture, which is not needed

    #For data analysis, we can return the scores
    return sum(human_scores), sum(bot_scores)

# simulate_game(6)
```

3.2 Simulation 1.5 - Penalties for blocked bots

Intuitively, Simulation 1 is expected to result in failure for humans. This raises the question: How can we influence this? Can we change the game more in their favor?
So we thought of a simple idea, where humans can block a bot from creating more content. To simulate this idea we introduce a penalty system for bots. If the human payoff is negative, meaning that he is not happy with the reels or posts generated by the AI, a penalty is applied to the bot payoff (new payoff = original payoff - penalty).
In Simulation 1.5, humans continue to interact with bots and vice versa. However, humans now have the ability to report bots. The more frequently a bot is reported/blocked, the greater the penalty it will get. This is described by a linear trend with a strength of two. When the penalty is equal to zero, the setup is the same as in Simulation 1.
The Code for the following simulation works on it's base as Simulation 1, but has some tweaks. On top of the tracking scores, we initialized in Lines 34 to 44 some additional variables. We defined the `penalty_strength` which describes the penalty factor. The more a bot gets reported, the higher the penalty should be. The parameter `report_prob` describes the probability that a bot gets reported. The `bot_pool` variable is an array that keeps track of all individual penalties, and the `num_bots` describes the number of bots in this game. Line 42 is used to initialize the array. The biggest difference to Simulation 1 lies between Lines 58 and 64, since there we check if an interaction warrants a report and also we calculate the penalty, and then update the score. The rest of the code remains the same.

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  #Human Users (H):
5  #   H1: Seek Authenticity
6  #   H2: Engage with Flow
7
8  #Bot Operators (B):
9  #   B1: Mimic Humanity
10 #   B2: Mass Produce
11
12 # Payoff-Matrix: [H1 vs B1, H1 vs B2, H2 vs B1, H2 vs B2]
13 payoffs = {
14     ('H1', 'B1'): (20, 10),
15     ('H1', 'B2'): (30, 0),
16     ('H2', 'B1'): (0, 20),
17     ('H2', 'B2'): (-20, 30)
18 }
19
20 # Number of rounds to simulate
21 rounds = 100
22
23 # Initial distribution (can be adjusted)
24 human_distribution = {'H1': 0.5, 'H2': 0.5}
25 bot_distribution = {'B1': 0.5, 'B2': 0.5}
26
27 def simulate_game(games):
28
29     #Tracking scores
30     human_scores = []
31     bot_scores = []
32
33     # Initialize penalty counts and strength
```

```python
    penalty_strength = 2  #linear factor: the higher, the harder
    report_prob = 0.3  # probability that a problematic interaction leads to a report

     # Generate a pool of bot instances with individual penalty counters
    bot_pool = []
    num_bots = 10  # Number of bots in the pool, can be adjusted

    # Initialize bot pool based on the number of bots and their distribution
    for _ in range(num_bots):
        bot_type = np.random.choice(['B1', 'B2'], p=[bot_distribution['B1'], bot_distribution['B2']])
        bot_pool.append({'type': bot_type, 'penalty_count': 0})

    for _ in range(rounds):
        # Randomly pick strategies
        human = np.random.choice(['H1', 'H2'], p=[human_distribution['H1'], human_distribution['H2']])

        # Select a random bot instance
        bot = np.random.choice(bot_pool)
        bot_type = bot['type']

        # Get payoffs based on the chosen strategies
        h_payoff, b_payoff = payoffs[(human, bot_type)]

        # Check if this interaction warrants a report
        if (human == 'H1' and bot_type == 'B2') or (human == 'H2' and bot_type == 'B2' and h_payoff < 0):
            if np.random.rand() < report_prob:
                bot['penalty_count'] += 1

        # Calculate penalty
        penalty = penalty_strength * bot['penalty_count']
        b_payoff -= penalty  # Apply penalty to bot's payoff

        # Update scores
        human_scores.append(h_payoff)
        bot_scores.append(b_payoff)

    # Plot results
    # Display the settings used in the simulation
    settings_text = (
        f"H1/H2: {human_distribution['H1']*100:.0f}% / {human_distribution['H2']*100:.0f}%\n"
        f"B1/B2: {bot_distribution['B1']*100:.0f}% / {bot_distribution['B2']*100:.0f}%\n"
    )

    # Adjusting the plot layout to include settings text
    plt.subplots_adjust(bottom=0.2)
    plt.gcf().text(0.01, 0.01, settings_text, fontsize=9, verticalalignment='bottom')

    # Plotting cumulative scores
    plt.plot(np.cumsum(human_scores), label='Human Payoff', color='red')
    plt.plot(np.cumsum(bot_scores), label='Bot Payoff', color='blue')

    # Adding labels and title to the plot
    plt.xlabel("Round")
    plt.ylabel("Cumulative Payoff")
    plt.title("Authenticity Signal Game     Simulation 1")          #Title displayed on the plot
    plt.suptitle(f"Number of Game: {games}" , fontsize=10)        #Subtitle displayed on the plot
    plt.legend()                                                  #adds legend to the plot
    plt.grid(True)                                                #adds grid to the plot


    plt.savefig(f"Results_Sim1-5/sim1-5_{games}.png")
    plt.clf()                                                     #deletes plot for next simulation

    # plt.show()                                                  #shows the picture, which is not needed
```

```

    #For data analysis, we can return the scores
    return sum(human_scores), sum(bot_scores)

simulate_game(6)
```

3.3 Simulation 2 - Ban blocked bots

Penalties and fines are effective, but do not solve the root of the issue. What a bot would do if he gets a lot of penalties is to open a new account and start again.
In the real world, one can get blocked only a limited amount of times, until the account is shut down/banned and can not be reactivated by creating a new account. This is what we want to recreate with Simulation 2.
The base of the following code is from Simulation 1 3.1. Lines 28 to 31 introduce a threshold and probability for each bot-type that describe how often a human needs to report a bot until he is blocked and with which probability a human of type H1 will recognize a bot and report it. In Lines 35/36 we additionally defined two arrays, one for each type of bot, and initialized those. They will act as trackers to track each bot and how often it is reported. The following Lines 39/40 are the "black List", where blocked bots will be stored.
The game starts the same as in the other two simulations, only that we need to filter an active bot that can interact with a human. If there is no active bot in it's type, we only add the human payoff taken from the matrix. Lines 78 to 82 describe the reporting part of the game. Since only H1 seeks true authenticity, only he is able to report bots for simplicity reasons. We increment the counter of a bot and, if necessary, block it. The rest of the code remains the same as in the games above.

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  #Human Users (H):
5  #   H1: Seek Authenticity
6  #   H2: Engage with Flow
7
8  #Bot Operators (B):
9  #   B1: Mimic Humanity
10 #   B2: Mass Produce
11
12 # Payoff-Matrix: [H1 vs B1, H1 vs B2, H2 vs B1, H2 vs B2]
13 payoffs = {
14     ('H1', 'B1'): (20, 10),
15     ('H1', 'B2'): (30, 0),
16     ('H2', 'B1'): (0, 20),
17     ('H2', 'B2'): (-20, 30)
18 }
19
20 # Number of rounds to simulate
21 rounds = 100
22
23 # Initial distribution (can be adjusted)
24 human_distribution = {'H1': 0.5, 'H2': 0.5}
25 bot_distribution = {'B1': 0.5, 'B2': 0.5}
26
27 # New Rule: Humans can report Bots
28 report_threshold_b1 = 8      # B1 will get blocked after 8 reports (more common)
29 report_threshold_b2 = 3      # B2 wwill get blocked after 3 reports (less common)
30 report_prob_b1 = 0.05        # H1 recognizes B1 with low probability
31 report_prob_b2 = 0.75        # H1 recognizes B2 with high probability
32
33 def simulate_game(games):
34     # Bots initialized with reports and blocked status
35     b1_bots = [{'id': f'B1_{i}', 'reports': 0, 'blocked': False} for i in range(10)]
36     b2_bots = [{'id': f'B2_{i}', 'reports': 0, 'blocked': False} for i in range(10)]
37
38     #Tracking blocked bots
39     blocked_b1 = []
40     blocked_b2 = []
41
42     #Tracking scores
43     human_scores = []
44     bot_scores = []
```

```python

    for _ in range(rounds):
        # Randomly pick strategies
        human = np.random.choice(['H1', 'H2'], p=[human_distribution['H1'], human_distribution['H2']])
        bot_type = np.random.choice(['B1', 'B2'], p=[bot_distribution['B1'], bot_distribution['B2']])

        # Select an active bot based on type
        if bot_type == 'B1':
            active_bots = [b for b in b1_bots if not b['blocked']] # Filter out blocked bots
            if not active_bots:                                    # If no active bots, humans get there penalties
                h_payoff, b_payoff = payoffs[(human, bot_type)]
                human_scores.append(h_payoff)
                continue
            bot = np.random.choice(active_bots)                     # Randomly select one of the active bots
            threshold = report_threshold_b1                         # Set threshold for reporting
            report_prob = report_prob_b1                            # Set reporting probability
            blocked_list = blocked_b1                               # List to track blocked B1 bots

        else:
            active_bots = [b for b in b2_bots if not b['blocked']]
            if not active_bots:
                h_payoff, b_payoff = payoffs[(human, bot_type)]
                human_scores.append(h_payoff)
                continue
            bot = np.random.choice(active_bots)
            threshold = report_threshold_b2
            report_prob = report_prob_b2
            blocked_list = blocked_b2

        # Standard-Payoffs
        h_payoff, b_payoff = payoffs[(human, bot_type)]

        # Human H1 can report Bots [with a certain probability]
        if human == 'H1' and np.random.rand() < report_prob:
            bot['reports'] += 1                                     # Increment the report count for the bot
            if bot['reports'] >= threshold:                         # Check if the bot has reached the reporting threshold
                bot['blocked'] = True                               # Block the bot if threshold is reached
                blocked_list.append(bot['id'])                      # Add the bot to the blocked list

        human_scores.append(h_payoff)
        bot_scores.append(b_payoff)



    # Plot results
    # Displaying the settings used in the simulation
    settings_text = (
        f"H1/H2: {human_distribution['H1']*100:.0f}% / {human_distribution['H2']*100:.0f}%\n"
        f"B1/B2: {bot_distribution['B1']*100:.0f}% / {bot_distribution['B2']*100:.0f}%\n"
        f"Report Prob B1/B2: {report_prob_b1*100:.0f}% / {report_prob_b2*100:.0f}%\n"
        f"Blocking Threshold B1/B2: {report_threshold_b1} / {report_threshold_b2}"
    )
    # Adjusting the plot layout to include the settings text
    plt.subplots_adjust(bottom=0.2)
    plt.gcf().text(0.01, 0.01, settings_text, fontsize=9, verticalalignment='bottom')

    # Plotting the cumulative scores
    plt.plot(np.cumsum(human_scores), label='Human Payoff', color='red')
    plt.plot(np.cumsum(bot_scores), label='Bot Payoff', color='blue')

```

```
    # Adding labels and title to the plot
    plt.xlabel("Round")
    plt.ylabel("Cumulative Payoff")
    plt.title("Authenticity Signal Game     Simulation 2")
    plt.suptitle(f"Number of Game: {games}" , fontsize=10)
    plt.legend()
    plt.grid(True)

    plt.savefig(f"Results_Sim2/sim2_{games}.png")

    # plt.show()    #shows the picture, which is not needed
    plt.clf()                                                    #deletes plot for next simulation

    #For data analysis, we can return the scores
    return sum(human_scores), sum(bot_scores)
```

3.4 Game Analysis

The following method can be used to analyze each simulation individually. To simplify the analysis, one only has to change the parameter "`games`", which represents the amount of games that are played. The parameter "`chosen` $\in [1, 2]$" represents which simulation we choose (Simulation 1.5 is only an intermediate step and is not considered for this analysis).
The basic idea of this analysis is to compare more than one game with each other and get an average across multiple stages.
More detailed: Lines 6 to 22 initialize variables, to define the number of games and the chosen simulation, as well as the analysing variables we want to compare:

- How often the humans win
- How often the bots win
- Their respective total scores

Lines 32 to 46 describe the games. Each game is separately executed and the scores are updated.
Lines 48 to 64 describe the analysis tool. It writes the following data points into a txt file:

- Number of Games
- Distributions chosen
- Respective total scores
- Humans/Bot win percentage
- Score Ratio

```
1  import simulation1 as sim1
2  import simulation2 as sim2
3  import simulation1_5 as sim1_5
4
5  # Change the followng lines to adjust the simulations
6  games = 500          # Here we define the number of games that we run
7  chosen = 2          # This variable can be used to choose which simulation to run, e.g., sim1 or sim2
8
9  # We define some variables for some statistics
10 human_wins = 0
11 bot_wins = 0
12
13 # We gather the total scores of the human and bot players
14 human_total_score = 0
15 bot_total_score = 0
16
17 # We create a mapping of the simulations to choose from, s.t. we only need to change one variable to switch between them
18 sim_map = {
19     1: sim1,
20     2: sim2,
```

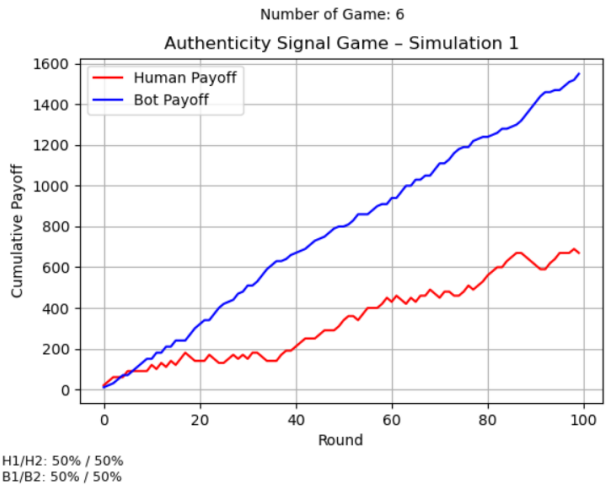
```python
    1_5: sim1_5
}

# We check if the chosen simulation is valid (Error handling)
if chosen not in sim_map:
    raise ValueError("chosen must be 1 or 2")

# We get the simulation object based on the chosen variable
sim = sim_map[chosen]

# We run the simulation for the defined number of games
for i in range(games):

    # We simulate the game and get the scores
    (human_score, bot_score) = sim.simulate_game(i)

    # We add the scores to the total scores
    human_total_score += human_score
    bot_total_score += bot_score

    # We check who won the game and add to the win counter
    if(human_score > bot_score):
        human_wins += 1
        print(f"Game {i+1}: Human wins with score {human_score} vs Bot score {bot_score}"
    )
    elif(human_score < bot_score):
        bot_wins += 1

# We print the results of the simulation, to analyse them
print(f"For {games} games, we get the ratio of human wins: {human_wins/games:.2f} and bot
     wins: {bot_wins/games:.2f}")
print(f"Total Human Score: {human_total_score}, Total Bot Score: {bot_total_score} which
    gives us a ratio of {human_total_score/bot_total_score:.2f}")

# We write the results to a file, so we can analyse them later
with open(f"simulation{chosen}_results.txt", "w") as file:

    file.write(f"Number of Games: {games}\n")

    file.write(f"Human Distribution: {sim.human_distribution}\n")
    file.write(f"Bot Distribution: {sim.bot_distribution}\n")

    file.write(f"Total Human Score: {human_total_score}\n")
    file.write(f"Total Bot Score: {bot_total_score}\n")
    file.write(f"Human Wins: {human_wins} ({100*human_wins/games:.2f}%)\n")
    file.write(f"Bot Wins: {bot_wins} ({100*bot_wins/games:.2f}%)\n")
    file.write(f"Score Ratio (Human/Bot): {human_total_score/bot_total_score:.2f}\n")
```

4 Results

In this subchapter we aim to find interesting phenomena, patterns and possibly convergences & Nash Equilibria of the simulations stated in chapter 3. We assume the constant payoff matrix which was described there as well. Based on this payoff matrix we can state that Humans have H1 as their dominant strategy, as this maximizes profit regardless of what strategy the bot chooses. The bot, however, has no dominant strategy, as the preferred strategy is dependent on the human's strategy choice. This means that there is no Nash Equilibrium for this payoff matrix, as that would mean there exists a stable strategy, s.t. no other player could improve their strategies, whilst all other players maintain their strategies.
In the following, we observe what happens to each simulation after 50 games, where each game has 100 rounds. We primarily focus on the parameter for the human distribution, as well as the additional parameters which are available for Simulations 1.5 and 2. By the construction of all simulations there is randomness and the results are non-deterministic; however, we attempt to show mostly reoccurring patterns after multiple executions as well as not focusing too much on edge cases.

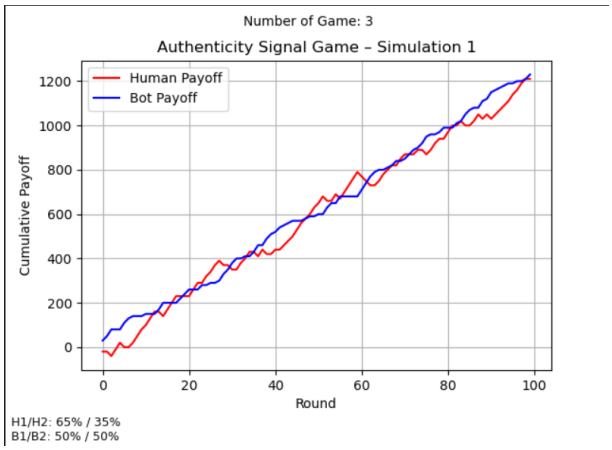
4.1 Base Game (Simulation 1)

For the first simulation, we decided to see what happens if both the bots' and humans' distribution is 50/50. We can state that the bots' payoff is much higher compared to the humans' payoff, with it growing in the long term almost linearly. This was almost always the case for all the games in this simulation, regardless of the human distribution of the strategies. The human payoff heavily deviates; however, the slope is on average almost half that of the bot.




```
Number of Games: 50
Human Distribution: {'H1': 0.5, 'H2': 0.5}
Bot Distribution: {'B1': 0.5, 'B2': 0.5}
Total Human Score: 37310
Total Bot Score: 74930
Human Wins: 0 (0.00%)
Bot Wins: 50 (100.00%)
Score Ratio (Human/Bot): 0.50
```

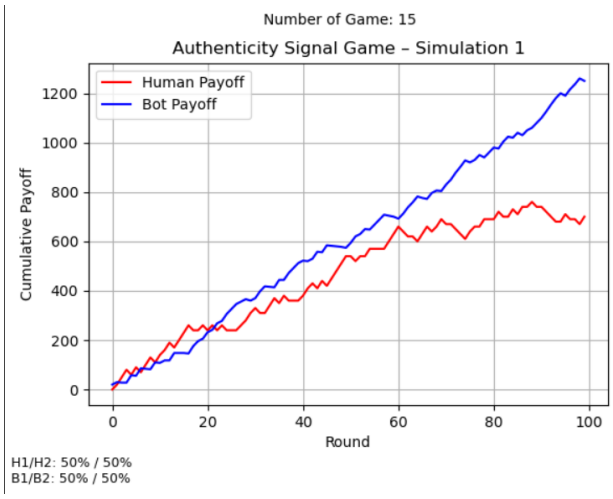
By increasing the probability for the human strategy H1 to 0.65 (and accordingly decreasing H2 to 0.35), we can state that the ratio of human payoff to bot payoff is roughly matched, despite the heavy deviations of the human payoff. This confirms that H1 is indeed the dominant strategy and pursuing H2 is not desirable at all. This is in contrary to the bot strategy distribution, as there is no dominant strategy and therefore changing this barely has any effect on human payoff.




```
Number of Games: 50
Human Distribution: {'H1': 0.65, 'H2': 0.35}
Bot Distribution: {'B1': 0.5, 'B2': 0.5}
Total Human Score: 62730
Total Bot Score: 60800
Human Wins: 25 (50.00%)
Bot Wins: 24 (48.00%)
Score Ratio (Human/Bot): 1.03
```

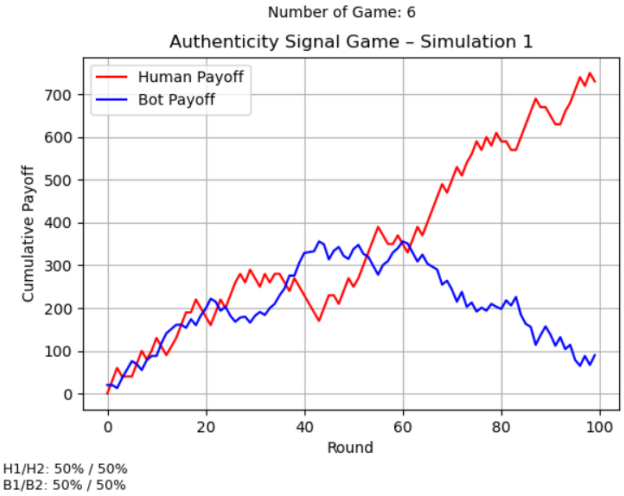
4.2 Game with Penalties (Simulation 1.5)

When running Simulation 1.5 with the default parameters, that is, a 50/50 distribution of both strategies for both B1/B2 and H1/H2, as well as a penalty strength/weight of 2 and a report probability of 0.3, we receive games with primarily the following phenomena:




```
Number of Games: 50
Human Distribution: {'H1': 0.5, 'H2': 0.5}
Bot Distribution: {'B1': 0.5, 'B2': 0.5}
Total Human Score: 37710
Total Bot Score: 67098
Human Wins: 0 (0.00%)
Bot Wins: 50 (100.00%)
Score Ratio (Human/Bot): 0.56
penalty_strength = 2
report_prob = 0.3
```

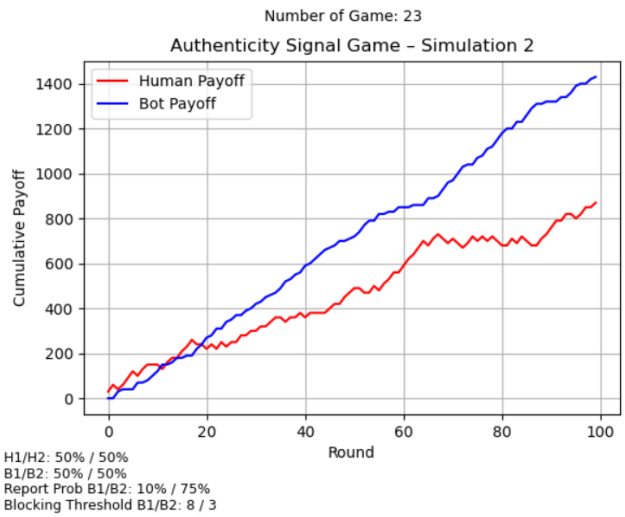
The behaviour is very similar to the first simulation where both human and bot distribution was 50%, however there is an increase in payoffs for humans, with sometimes 1 or even 2 games won by them (compared to 0 before). Although the bots' payout throughout the progression of the game is not constantly increasing, the long-term trend is still linear. However, what occurs if solely the penalty_strength increases? If the penalty_strength is increased to 7, there is an increase in deviations, and both payoffs are not really stable, especially the human payoff. See, for example, an extreme game depicted in the figure below. There are cases where already early on in the game, dominance of one side is established and maintained. The threshold of almost equal payoffs between both entities (assuming the other parameters stay constant) is around 8. The parameter for report probability has only minor affects for low penalty_strength (<5). For higher penalty strengths, the report probability can, however, have a big influence as well and drastically increase human payoffs depending on the selection.




```
Number of Games: 50
Human Distribution: {'H1': 0.5, 'H2': 0.5}
Bot Distribution: {'B1': 0.5, 'B2': 0.5}
Total Human Score: 36020
Total Bot Score: 42528
Human Wins: 16 (32.00%)
Bot Wins: 34 (68.00%)
Score Ratio (Human/Bot): 0.85
penalty_strength = 7
report_prob = 0.3
```

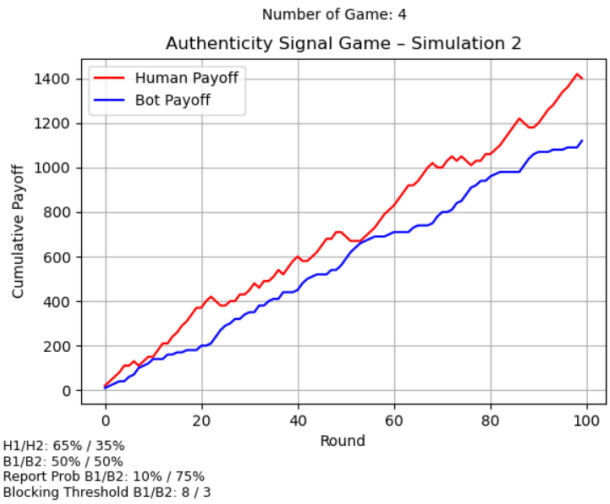
4.3 Game with Reports/Bans (Simulation 2)

By introducing blocking & reports in Simulation 2, we notice that while the bot payoff does increase relatively linearly, occasionally it stagnates due to the blockages. Even in the 50/50 split of human and bot distributions with reporting and blocking, humans have a higher payoff compared to both simulations before.




```
Number of Games: 50
Human Distribution: {'H1': 0.5, 'H2': 0.5}
Bot Distribution: {'B1': 0.5, 'B2': 0.5}
Total Human Score: 37760
Total Bot Score: 74950
Human Wins: 2 (4.00%)
Bot Wins: 48 (96.00%)
Score Ratio (Human/Bot): 0.50

report_threshold_b1: 8
report_threshold_b2: 3
report_prob_b1: 0.1
report_prob_b2: 0.75
```

We can state that the threshold for even payoffs is around 65/35, i.e. 65% of humans pursuing H1 and 35% H2. It is important to note that in this case there is a high variance, i.e. the payoff of humans varies extremely. Even slight increases in the probability of the H1 strategy significantly increase human rewards compared to bots.




```
Number of Games: 50
Human Distribution: {'H1': 0.65, 'H2': 0.35}
Bot Distribution: {'B1': 0.5, 'B2': 0.5}
Total Human Score: 63460
Total Bot Score: 59720
Human Wins: 31 (62.00%)
Bot Wins: 19 (38.00%)
Score Ratio (Human/Bot): 1.06

report_threshold_b1: 8
report_threshold_b2: 3
report_prob_b1: 0.1
report_prob_b2: 0.75
```

Finally, we can state that reporting obvious (strategy B2), as well as the not so obvious (strategy B1) bot activity leads to high payoffs for humans, due to the report threshold of both bot strategies. In addition, critical thinking and evaluation can make for a more authentic and human internet experience. In practice, there exist still a lot of edge cases, such as False Negatives or False Positives, that were attempted to be conveyed through probabilistic measures. Furthermore, the increase of LLM's makes it very difficult to discern the real from fake, especially if solely text is involved.

5 Conclusion

From our examination of the Dead Internet Theory through many domains of game theory and multiple simulations we have revealed interesting and central insights into the interactions between humans and bots on the Web. Our understanding of the factors that influence all this and the outcomes of multiple different games has been greatly improved. The results of the strategic game dynamics between AI and people have costly implications on the future of the Internet and the world.

Authenticity plays a huge role in creating trust of us humans. We rely on our judgement to decide whether the content is useful to us or not. And mass-produced AI products are not what we want, so bots try to mimic reality. This leads to a game of deception and detection, or, as we call it, the Authenticity Signal Game.

From our three simulations, we have concluded that both humans and bots can adapt strategically and rationally to get a positive payoff. However, there is no Nash equilibrium where a stable strategy exists for both parties. H1 is the dominant strategy for humans in all scenarios. In Simulation 1 the humans only get half the payoff to the bots and have zero chance of winning (for a 50/50 ratio of B1/B2 and H1/H2). It gets more interesting in Simulation 1.5, where the human has a small chance of winning (depending on how high the penalty_strength is set), and their payoff increases. For the final simulation, the results show a staggeringly high payoff and win rate for humans, especially with increased reporting threshold and probability, i.e. choosing to inspect content and report a bot, if found, increases our chances of winning a lot.

It is never guaranteed that there is no manipulative behaviour happening from artificially generated content, but that all online news and posts are controlled by the government is rather improbable regarding our statistics. This questions the truthfulness of the Dead Internet Theory and its fundamentals. Although our results do suggest that external intervention may be necessary to shift the outcome of the game in favour of authenticity and us humans in the long run.

Next steps would be to extend our model to multiple players, different types of humans, various strategies, and possibly even learning behaviours of bots. Real-world data could definitely help to find patterns in this labyrinth of possible scenarios. Understanding these scenarios and their outcomes is essential for the web to remain a space where authentic, creative and genuine content thrives and no manipulative algorithms ruin our perception of reality.

6 Bibliography

References

[1] Wikipedia contributors. Dead internet theory, 2024. Accessed: 2025-06-11.

[2] Manfred J. Holler. Classical, modern and new game theory. *Yale, Law*, 2001-8-4.

[3] Cristina Bicchieri. Rationality and game theory. *Oxford Handbook*, 2009-9-2.

[4] Joel Tan and Cyril Revel. Green-beard genes in the evolution of cooperation. 2024.

[5] Simon Jantschgi. Evolutionary game theory. Powerpoint Slides, Lecture "Controversies in Game Theory 2025".

[6] Theo Tsaousides Ph.D. What it means to be truly authentic. *Psychology Today*, 2023.

[7] Erica R. Bailey and Aharon Levy. Are you for real? perceptions of authenticity are systematically biased and not accurate. *Psychological Science*, 33(5):798–815, 2022. PMID: 35482890.

[8] Yunhao Zhang and Renee Gosline. Human favoritism, not ai aversion: People's perceptions (and bias) toward generative ai, human experts, and human-gai collaboration in persuasive content generation. *MIT*, 2023-5-20.

[9] Su-I Lu Janet Chen and Dan Vekhter. Zero-sum-games. *CS Stanford*, 1999.

[10] Mehmet S. Ismail. Exploring the constraints on artificial general intelligence: A game-theoretic model of human vs machine interaction. *Mathematical Social Sciences*, 129:70–76, 2024.

7 Appendix

GitHub Repository: `https://github.com/SmallPandaWorld/Contreversal_Game-Theory`